MARATHWADA MITRA MANDAL'S

COLLEGE OF ENGINEERING, KARVENAGAR, PUNE – 52

An Autonomous Institute affiliated to SPPU

NAAC A++ Accredited | NBA Accredited

A SPOS MINI PROJECT REPORT

ON

"FRESHFLOW"

Production Scheduling Simulator using Non-preemptive SJF and Preemptive SJF (SRTF)

Submitted by

Ashlesha Humne – B24CE1031

Dnyaneshwari Thorat – B24CE1035

Isha Patil – B24CE1049

Under the guidance of

Dr. Neha Jain

Department of Computer Engineering

Class: TY-I, Semester I | Academic Year 2026–27

Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

CERTIFICATE

This is to certify that Students Ashlesha Humne(B24CE1031),Dnyaneshwari Thorat(B24CE1035),Isha Patil(B24CE1049) of class TY Computer Engineering have successfully completed their project work on “FRESHFLOW” at MARATHWADA MITRA MANDAL COLLEGE OF ENGINEERING in the partial fulfillment of the Graduate Degree course in T.E at theDepartment of Computer Engineering in the academic Year 2026-2027 Semester –I.

Dr. Neha Jain
Project Guide

Dr. S. S. Shiravale
Head of Department, Computer Engineering

ACKNOWLEDGEMENT

I feel great pleasure in expressing my deepest sense of gratitude and sincere thanks to my guide **Dr. Neha Jain** for their valuable guidance during the Project work, without which it would have been a very difficult task. I have no words to express my sincere thanks for valuable guidance, extreme assistance and cooperation extended to all the Staff Members of my Department.

This acknowledgement would be incomplete without expressing my special thanks to **Dr. S.S Shiravale,** Head of the Department (Computer Engineering) for their support during the work.

I would also like to extend my heartfelt gratitude to my Principal, **Dr. Sachin Sakhare** who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least I would like to thank all the Teaching, Non- Teaching staff members of my Department, my parents and my colleagues who helped me directly or indirectly for completing this Project successfully.

ABSTRACT

FreshFlow is a browser-based production scheduling simulator developed to demonstrate Shortest Job First (SJF) CPU scheduling in two forms: Non-preemptive SJF and Preemptive SJF, also known as Shortest Remaining Time First (SRTF). In a production environment, tasks may arrive at different times and require different processing durations. Selecting an appropriate execution order can reduce waiting and turnaround times, and a visual simulator makes these operating-system concepts easier to understand.

The application allows users to register production tasks by entering a task identifier, task name, arrival time, and burst time. The same task list can be scheduled using either algorithm. In Non-preemptive SJF, the scheduler selects the available task with the smallest burst time and allows it to finish without interruption. In SRTF, the scheduler selects the available task with the smallest remaining processing time and may preempt the running task when a shorter job arrives. FreshFlow displays a Gantt chart, completion time, turnaround time, waiting time, average waiting time, average turnaround time, and makespan. It also supports a side-by-side algorithm comparison and CSV export, with a print option for saving a report as PDF.

The front end is implemented using HTML, CSS, and JavaScript and runs in a modern web browser without requiring a server or database. The project demonstrates process scheduling, arrival-time handling, preemption, performance measurement, and user-interface design. The simulator is intended for educational use; its output depends on the input tasks and is not a claim that one algorithm is best for every workload.

Keywords: SPOS, CPU scheduling, Shortest Job First, SJF, Shortest Remaining Time First, SRTF, Gantt chart, waiting time, turnaround time.

'Welfare of Masses'
Est.1967

TABLE OF CONTENTS

Abstract

Chapter 1: Introduction

1.1 Project Overview

1.2 Problem Definition

1.3 Motivation

1.4 Objectives

1.5 Scope of the Project

Chapter 2: Theoretical Background

2.1 Selected SPOS Concept

2.2 Relevant Terminology

2.3 Algorithms / Techniques

2.4 Technologies and Tools Used

Chapter 3: System Analysis and Design

3.1 Functional Requirements

3.2 Hardware and Software Requirements

3.3 System Architecture / Flowchart

3.4 Design Considerations

3.5 Algorithm / Pseudocode

Chapter 4: Implementation

4.1 Implementation Approach

4.2 Module Description

4.3 Important Source Code with Explanation

4.4 Execution Procedure

4.5 Input and Output Screenshots

Chapter 5: Testing and Results

5.1 Test Cases

5.2 Actual Results

5.3 Result Analysis and Observations

5.4 Limitations

Conclusion

Future Scope

References

Note: Page numbers can be updated in Word after opening the document, if required.



CHAPTER 1: INTRODUCTION

1.1 Project Overview

FreshFlow is an interactive web application that simulates CPU scheduling for production tasks. It uses a food-processing and packaging theme to make scheduling examples approachable. Users enter task details and select one of two scheduling methods: Non-preemptive Shortest Job First (SJF) or Preemptive Shortest Remaining Time First (SRTF). The application then calculates scheduling metrics and displays the execution order as a Gantt chart.

Although the interface uses production terminology, arrival time and burst time represent the standard process attributes used in operating-system scheduling. The simulator is designed to support learning, experimentation, and comparison rather than to control actual industrial machinery.

1.2 Problem Definition

CPU scheduling determines which ready process receives the processor next. Different algorithms produce different execution orders and therefore different waiting and turnaround times. Manual calculation becomes error-prone when processes arrive at different times or when preemption occurs. The problem addressed by FreshFlow is to provide a simple, visual, and repeatable way to enter a set of tasks, apply both SJF variants, and compare their outcomes using consistent metrics.

1.3 Motivation

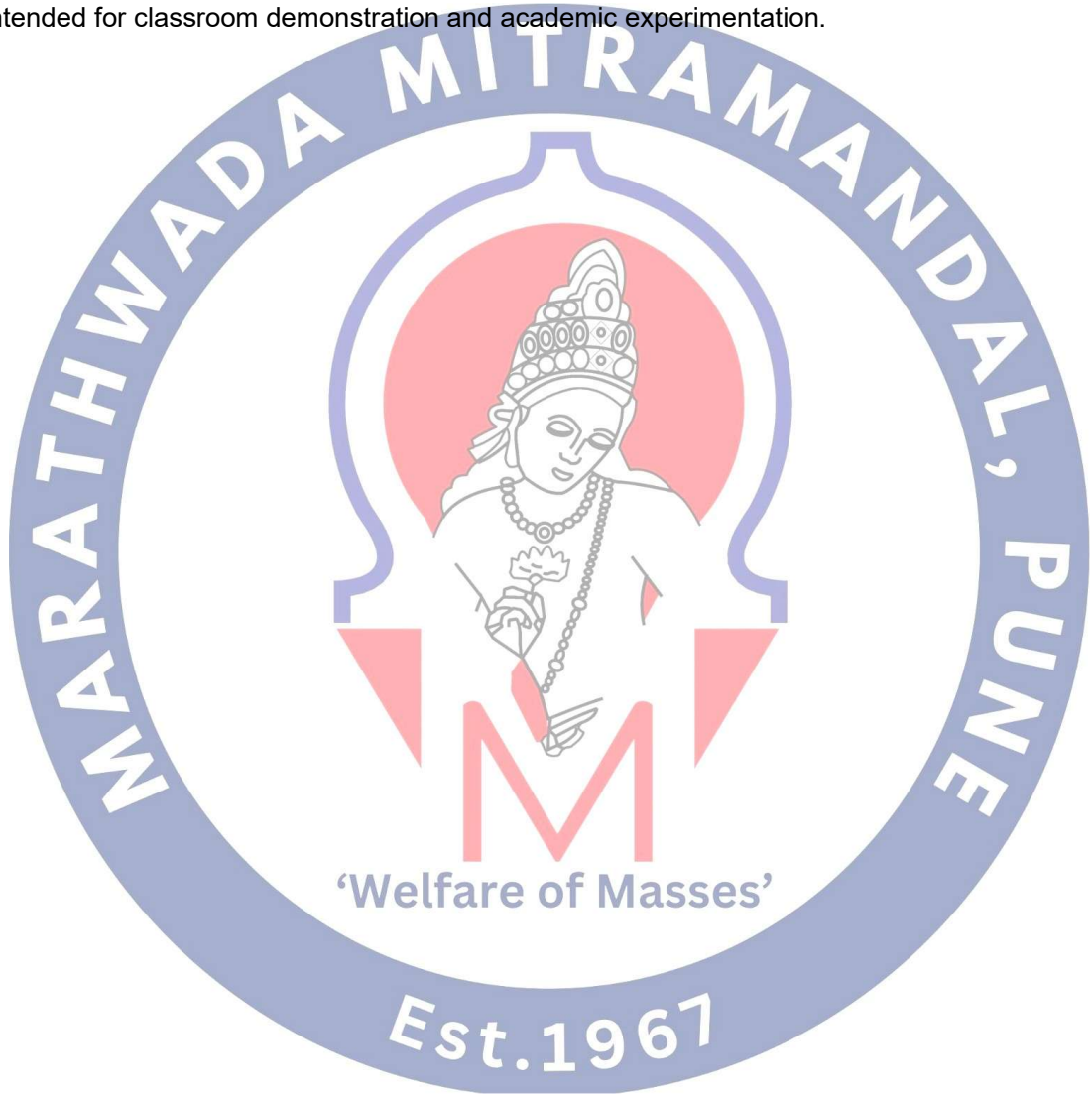
SJF and SRTF are important scheduling algorithms in SPOS, but their behavior can be difficult to follow from tables alone. A visual tool helps learners see when a task starts, finishes, waits, or is interrupted. FreshFlow also enables users to change task inputs and immediately observe how arrival and burst times affect the schedule.

1.4 Objectives

- Develop a simple browser-based scheduling simulator with a clean dashboard.
- Accept task ID, task name, arrival time, and burst time.
- Implement Non-preemptive SJF and Preemptive SJF (SRTF).
- Display a Gantt chart showing execution intervals and idle time.
- Calculate completion time, turnaround time, waiting time, average metrics, and makespan.
- Use the same task set to compare both algorithms fairly.
- Validate basic input conditions and support CSV export and printing.

1.5 Scope of the Project

The project covers a single-processor scheduling model with independent tasks, non-negative integer arrival times, and positive integer burst times. It displays calculated results in a browser and can export task-level results to CSV. It does not model multiple processors, priorities, I/O blocking, context-switch overhead, deadlines, or real-time industrial control. The application is intended for classroom demonstration and academic experimentation.



CHAPTER 2: THEORETICAL BACKGROUND

2.1 Selected SPOS Concept

The selected concept is CPU/process scheduling. An operating system maintains processes that are ready to execute and uses a scheduling policy to choose the next process. Scheduling policies influence responsiveness and resource utilization. SJF chooses the available process with the smallest total CPU burst, while SRTF chooses the process with the smallest remaining burst and permits preemption.

2.2 Relevant Terminology

Term	Meaning
Process / task	A unit of work to be scheduled. In FreshFlow it is shown as a production task.
Arrival time (AT)	Time at which a task becomes available for execution.
Burst time (BT)	Total processing time required by a task.
Remaining time	Processing time still required by a task during SRTF.
Completion time (CT)	Time at which a task finishes execution.
Turnaround time (TAT)	Total time from arrival until completion: TAT = CT − AT.
Waiting time (WT)	Total time spent waiting: WT = TAT − BT.
Preemption	Interrupting a running task to execute another task.
Makespan	Time at which all tasks have completed, including any initial idle interval.
Gantt chart	Timeline showing which task executes during each interval.

2.3 Algorithms / Techniques

2.3.1 Non-preemptive SJF

At each decision point, the scheduler considers all tasks that have arrived and have not completed. It selects the task with the smallest burst time. Once selected, that task runs to completion. If no task is ready, the processor remains idle until the next arrival. Ties are resolved consistently by arrival time and then input order in this implementation.

2.3.2 Preemptive SJF (SRTF)

SRTF selects the ready task with the smallest remaining time. Execution continues until the task completes or a new task arrives that changes the shortest-remaining-time choice. The current task may therefore be interrupted and resumed later. When no task is ready, the timeline records an idle interval.

2.3.3 Performance formulas

Completion time (CT) is the finish time of each task. Turnaround time is calculated as TAT = CT − AT. Waiting time is WT = TAT − BT. Average waiting time = sum of all waiting times / number of tasks. Average turnaround time = sum of all turnaround times / number of tasks. These formulas assume each task has one CPU burst and no I/O blocking.

2.4 Technologies and Tools Used

Technology / tool	Purpose
HTML5	Defines the dashboard structure, forms, navigation, tables, and report sections.
CSS3	Provides layout, colors, responsive styling, chart appearance, and light/dark theme.
JavaScript	Validates inputs, executes scheduling algorithms, calculates metrics, and updates the interface.
Modern web browser	Runs the application locally.
Visual Studio Code (optional)	Used to edit project files.
CSV / browser print	Exports results for spreadsheet analysis or printing to PDF.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Functional Requirements

- The system shall display an overview dashboard and navigation between sections.
- The user shall add tasks with unique task IDs, names, arrival times, and positive burst times.
- The system shall reject duplicate task IDs and invalid time values.
- The user shall select Non-preemptive SJF or Preemptive SJF (SRTF).
- The scheduler shall account for task arrival times and idle intervals.
- The system shall calculate CT, TAT, WT, average waiting time, average turnaround time, and makespan.
- The system shall display a Gantt chart and a result table.
- The user shall compare the two algorithms on the same task list.
- The system shall export a CSV report and provide a print option.
- The user shall be able to remove tasks or load sample tasks.

3.2 Hardware and Software Requirements

Type	Minimum / recommended requirement
Processor	Any modern dual-core processor or equivalent
Memory	4 GB RAM or more recommended
Storage	A few megabytes for the project files
Operating system	Windows, macOS, or Linux
Browser	Recent version of Chrome, Edge, Firefox, or another standards-compliant browser
Development editor	Visual Studio Code or any text editor (optional)
Dependencies	No server-side dependencies; internet is only needed if externally hosted fonts are used.

3.3 System Architecture / Block Diagram / Flowchart

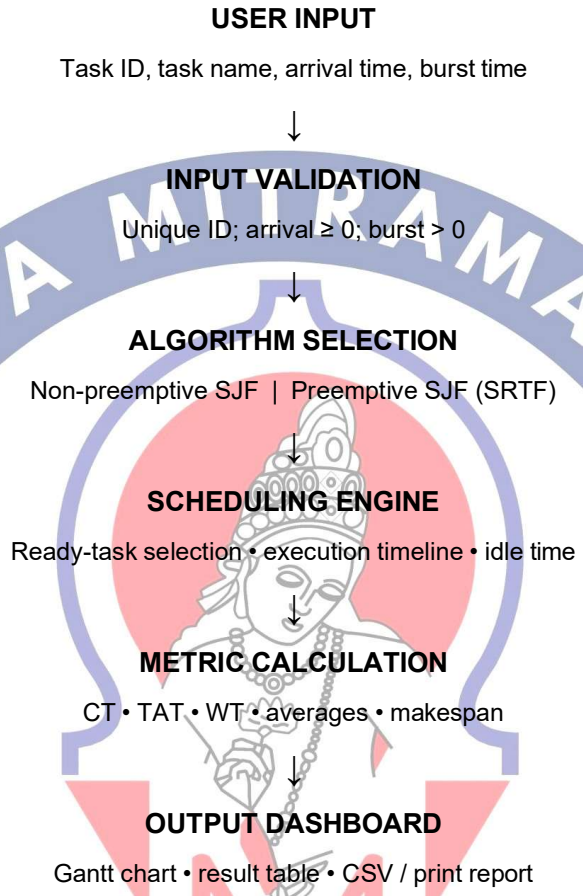


Figure 3.1: FreshFlow system flow. The browser interface collects task data, validates it, passes a copy of the task list to the selected scheduling algorithm, calculates performance metrics, and renders the schedule and report.

3.4 Design Considerations

The application separates task management, scheduling logic, visualization, and export actions. Both algorithms operate on the same input records, but each run uses a copy so that the original burst values remain unchanged. The timeline is represented as consecutive segments with start and end times. Adjacent segments for the same task may be merged for a cleaner Gantt chart. The design assumes a single processor and zero context-switch overhead.

3.5 Algorithm / Pseudocode

Algorithm A: Non-preemptive SJF

1. Initialize current time to 0 and mark every task as unfinished.

2. Find all unfinished tasks whose arrival time is less than or equal to current time.
3. If no task is ready, move current time to the next arrival and record an idle segment.
4. Otherwise, select the ready task with the smallest burst time; break ties consistently.
5. Run the selected task for its complete burst time and record its start and end times.
6. Set its completion time and mark it finished.
7. Repeat until every task has completed.
8. Calculate TAT = CT − AT and WT = TAT − BT for each task; calculate averages.

Algorithm B: Preemptive SJF (SRTF)

9. Initialize remaining time of each task to its burst time; set current time to 0.
10. Find all arrived tasks with remaining time greater than zero.
11. If none are ready, record idle time and advance to the next task arrival.
12. Select the ready task with the smallest remaining time.
13. Run it until it completes or the next arrival time, whichever comes first.
14. Reduce its remaining time by the executed duration and record the segment.
15. If its remaining time becomes zero, record its completion time; otherwise reconsider the ready queue at the next decision point.
16. Repeat until all tasks finish, then calculate TAT, WT, averages, and makespan.

MARATHWADA MITRAMANDAL, PUNE
'Welfare of Masses'
Est.1967

CHAPTER 4: IMPLEMENTATION

4.1 Implementation Approach

FreshFlow is implemented as a client-side web application using three files: index.html, style.css, and script.js. HTML defines the page structure, CSS handles the dashboard presentation, and JavaScript manages the task array, input validation, scheduling, result rendering, comparison, and export. The application can be opened directly in a browser; using a local development server such as Visual Studio Code Live Server is optional.

For each run, the scheduling function copies the input tasks and maintains a remaining-time value. Non-preemptive SJF selects by burst time and completes the selected task in one run. SRTF selects by remaining time and advances only to the next arrival or completion event. Once completion times are available, the application derives turnaround and waiting times using the standard formulas.

4.2 Module Description

Module	Responsibility
Dashboard and navigation	Shows overview cards and opens task, schedule, and report pages.
Task management	Adds, removes, clears, and reloads sample task records.
Input validation	Checks required values, unique IDs, non-negative integer arrivals, and positive integer bursts.
Non-preemptive SJF engine	Chooses the ready task with the smallest burst and runs it to completion.
SRTF engine	Chooses the ready task with the smallest remaining time and supports preemption.
Metrics calculator	Calculates CT, TAT, WT, average values, and makespan.
Gantt chart renderer	Shows execution segments and idle periods on a timeline.
Report and export	Compares both algorithms, downloads CSV, and opens browser print.

4.3 Important Source Code with Explanation

The following excerpts describe the central scheduling logic. They are illustrative core snippets; the complete runnable implementation is contained in the project's index.html, style.css, and script.js files.

Waiting and turnaround calculations

```
turnaround = completionTime - arrivalTime;
 waiting = turnaround - burstTime;
```

Turnaround time measures the total time a task spends in the system. Waiting time is the part of turnaround time not spent executing.

Ready-task selection

```
const ready = jobs
   .filter(job => job.remaining > 0 && job.arrival <= time)
   .sort((a, b) => a.remaining - b.remaining
            || a.arrival - b.arrival
            || a.originalIndex - b.originalIndex);
```

For SRTF, the ready queue is ordered by remaining processing time. Arrival time and original input order provide deterministic tie-breaking.

SRTF execution interval

```
const duration = Math.min(
   current.remaining,
   nextArrivalTime - currentTime
 );
 currentTime += duration;
 current.remaining -= duration;
```

The SRTF engine advances to the next arrival or task completion. It then re-evaluates which ready task has the shortest remaining time.

4.4 Execution Procedure

1. Create a folder named FreshFlow_SJF_Project.
2. Save the three application files as index.html, style.css, and script.js in the same folder.
3. Open index.html in a modern browser, or open the folder in Visual Studio Code and launch index.html with Live Server.
4. Use Production tasks to add tasks or click Load sample tasks.
5. Open SJF schedule and choose Non-preemptive SJF or Preemptive SJF (SRTF).
6. Click Run schedule and review the Gantt chart and results table.
7. Use Reports & export to compare both algorithms, download CSV, or print/save the report as PDF.

4.5 Input and Output Screenshots

Insert actual screenshots of the running application in this section before submission. Recommended screenshots are listed below; placeholders are intentionally provided because screenshots must come from the executed project.

Figure 4.1: FreshFlow overview dashboard with task statistics.

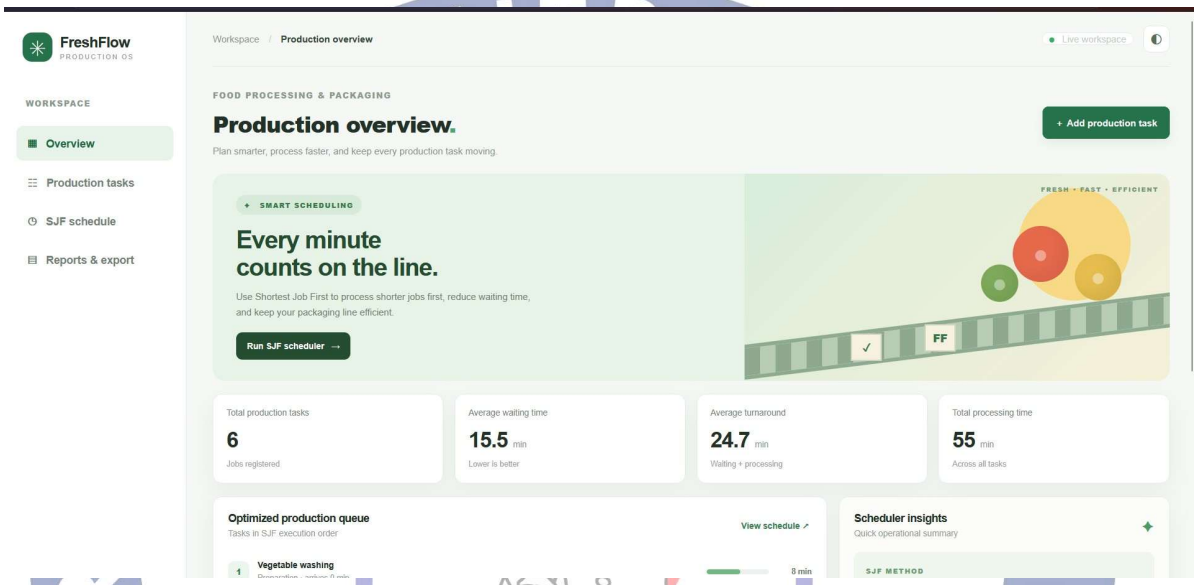


Figure 4.2: Production task form and registered task list.

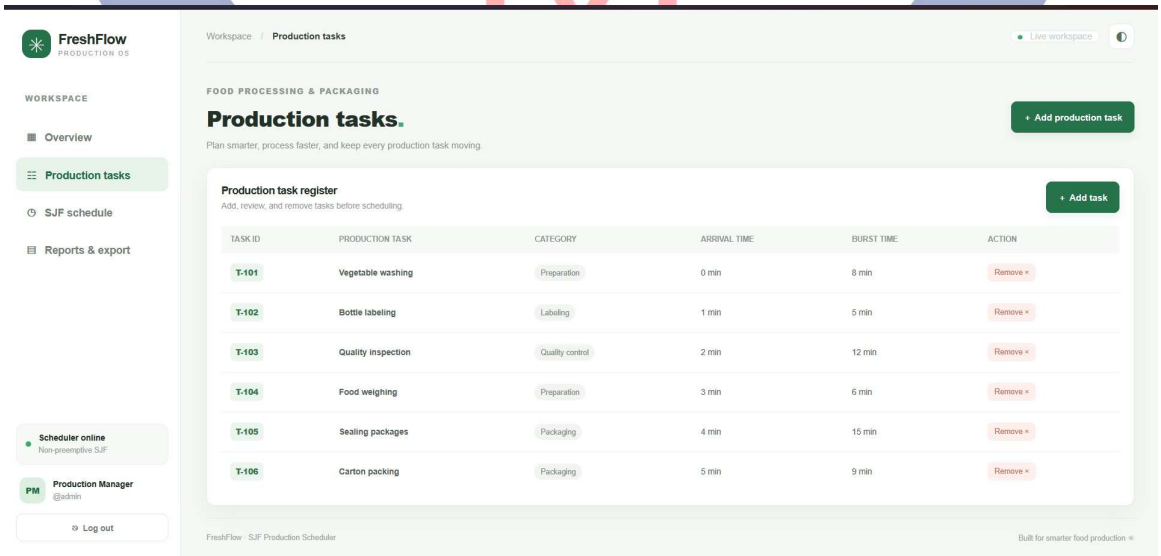


Figure 4.3: Non-preemptive SJF schedule with Gantt chart and results.

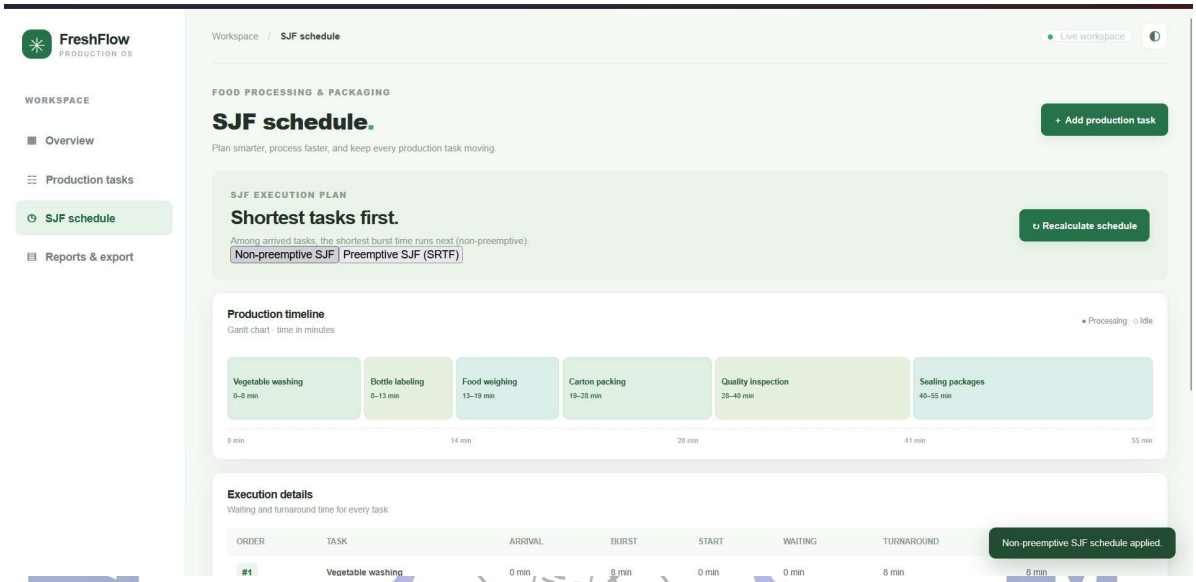


Figure 4.4: Preemptive SJF (SRTF) schedule showing a preemption, if present in the selected inputs.

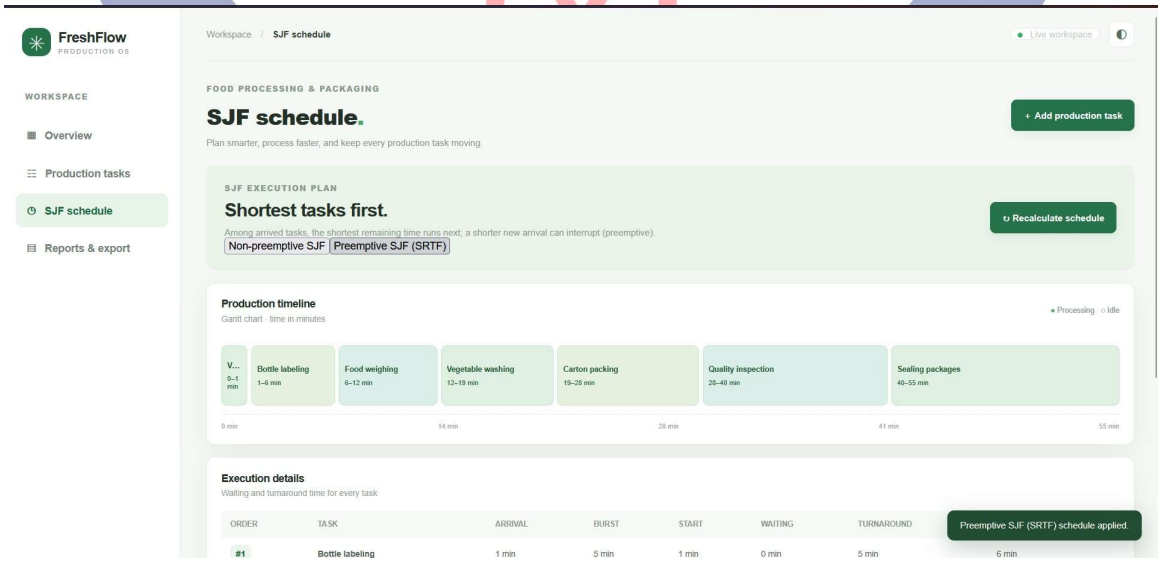
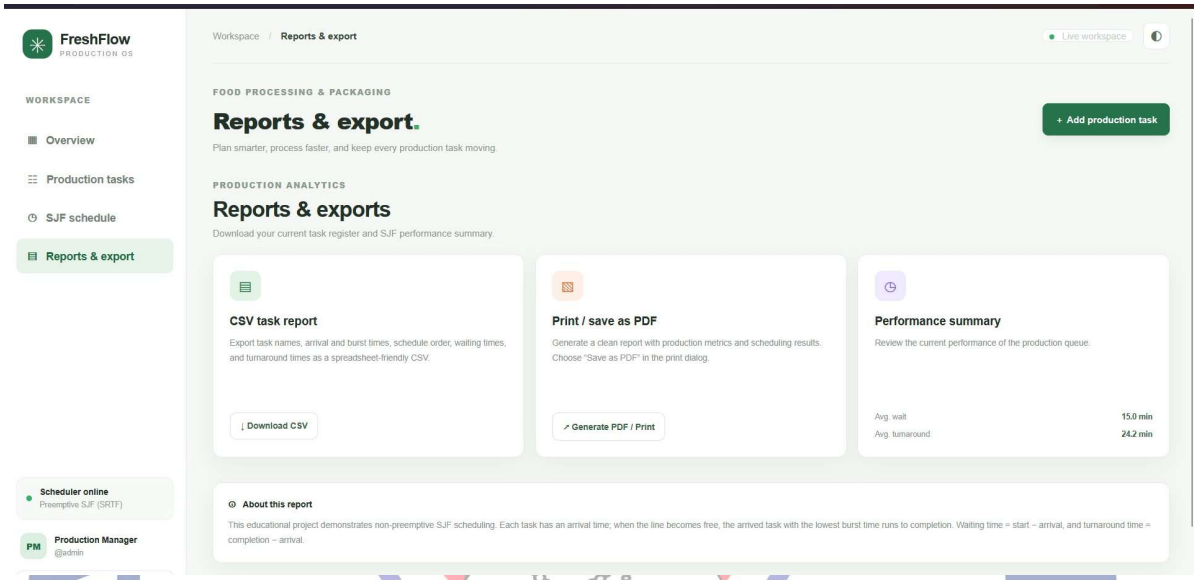


Figure 4.5: Reports page showing algorithm comparison and export options.



MARATHWADA MITRAMANDAL PUNE
'Welfare of Masses'
Est.1967

CHAPTER 5: TESTING AND RESULTS

5.1 Test Cases

Test Case ID	Input / Scenario	Expected Output	Actual Output	Status
TC01	Load the default sample tasks.	Six tasks appear in the task list.	To be recorded after execution.	Pending verification
TC02	Add a task with unique ID, arrival 0, burst 4.	Task is added to the list.	To be recorded after execution.	Pending verification
TC03	Add a duplicate task ID.	System rejects duplicate ID.	To be recorded after execution.	Pending verification
TC04	Enter negative arrival time.	Input is rejected by validation.	To be recorded after execution.	Pending verification
TC05	Enter burst time 0.	Input is rejected; burst must be positive.	To be recorded after execution.	Pending verification
TC06	Run Non-preemptive SJF with valid tasks.	Gantt chart and metrics are displayed.	To be recorded after execution.	Pending verification
TC07	Run SRTF with a shorter task arriving while another runs.	Scheduler may preempt the running task.	To be recorded after execution.	Pending verification
TC08	No task is ready at time 0 because all arrivals are later.	Gantt chart includes initial idle time.	To be recorded after execution.	Pending verification

TC09	Export current schedule to CSV.	CSV file downloads with task-level metrics.	To be recorded after execution.	Pending verification
TC10	Remove a task and rerun scheduling.	Removed task is excluded from results.	To be recorded after execution.	Pending verification

5.2 Actual Results

The application is designed to display a completion time, turnaround time, and waiting time for each valid task, along with average waiting time, average turnaround time, makespan, and a Gantt chart. Actual execution results and screenshots should be recorded after running the application in the target browser. The values below are a manually calculated illustrative example, not a claim of verified application execution.

Illustrative calculation example

Consider three tasks: T1 arrives at 0 with burst 8; T2 arrives at 1 with burst 4; T3 arrives at 2 with burst 2.

Task	Arrival	Burst	Non-preemptive completion	Non-preemptive TAT	Non-preemptive WT
T1	0	8	8	8	0
T2	1	4	14	13	9
T3	2	2	10	8	6

For this example, the non-preemptive schedule is T1 (0–8), T3 (8–10), T2 (10–14). Average waiting time = (0 + 9 + 6) / 3 = 5.00 time units; average turnaround time = (8 + 13 + 8) / 3 = 9.67 time units.

Task	Arrival	Burst	SRTF completion	SRTF TAT	SRTF WT
T1	0	8	14	14	6
T2	1	4	7	6	2
T3	2	2	4	2	0

For SRTF, one valid timeline is T1 (0–1), T2 (1–2), T3 (2–4), T2 (4–7), T1 (7–14). Average waiting time = (6 + 2 + 0) / 3 = 2.67 time units; average turnaround time = (14 + 6 + 2) / 3 = 7.33 time units. These values illustrate the formulas and must be checked against the implementation before being reported as actual test output.

5.3 Result Analysis and Observations

SJF variants make decisions using processing-time information, but they differ in whether a running task can be interrupted. Non-preemptive SJF is straightforward and avoids preemption, but a long task that starts may delay a later short task. SRTF can improve waiting time for short jobs when they arrive during a long job, at the cost of more task switches. Results depend on arrival and burst times; neither variant guarantees the lowest waiting time for every possible workload. Comparing both algorithms on the same input set makes the observed difference meaningful.

5.4 Limitations

- The model assumes one processor and one CPU burst per task.
- Burst times are assumed to be known in advance.
- Context-switch overhead is not included.
- I/O blocking, process priorities, deadlines, and multiprocessor scheduling are not modeled.
- The task list is held in browser memory; refreshing the page may reset changes.
- CSV export is supported; PDF output uses the browser print dialog rather than a dedicated PDF-generation engine.
- The project is an educational simulator and does not schedule real production equipment.

CONCLUSION

FreshFlow demonstrates two important operating-system CPU scheduling algorithms through an interactive production-themed interface. It accepts task arrival and burst times, schedules tasks using Non-preemptive SJF or Preemptive SJF (SRTF), and presents the execution timeline and standard performance metrics. The common input list and comparison feature help users observe how preemption and task-arrival order affect waiting and turnaround time. The project connects SPOS theory with practical implementation using HTML, CSS, and JavaScript. Before final academic submission, the team should execute all test cases, replace pending test outcomes with observed results, and insert genuine application screenshots.

FUTURE SCOPE

- Add First Come First Served (FCFS), Round Robin, Priority, and other scheduling algorithms.
- Provide automated comparison charts for average waiting time and turnaround time.
- Persist task lists and reports using local storage or a database.
- Allow configurable context-switch overhead and decimal time units.
- Add downloadable PDF generation and import of tasks from CSV.
- Improve accessibility and add more detailed step-by-step animation of scheduling decisions.
- Add multi-processor and priority-based scheduling as advanced learning modes.

REFERENCES

[1] Silberschatz, A., Galvin, P. B., and Gagne, G. Operating System Concepts. Wiley. Refer to the CPU scheduling chapter in the edition prescribed for the course.

[2] Stallings, W. Operating Systems: Internals and Design Principles. Pearson. Refer to the process management and processor scheduling sections.

[3] MDN Web Docs. HTML: HyperText Markup Language. https://developer.mozilla.org/en-US/docs/Web/HTML

[4] MDN Web Docs. CSS: Cascading Style Sheets. https://developer.mozilla.org/en-US/docs/Web/CSS

[5] MDN Web Docs. JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[6] FreshFlow project implementation files: index.html, style.css, and script.js (project source).